

MiniRISC 2026 Assignment Sequence

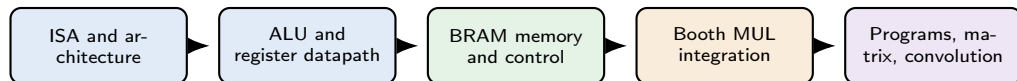
Plan with Dates, Modules and Flowcharts

Computer Organization and Architecture Laboratory

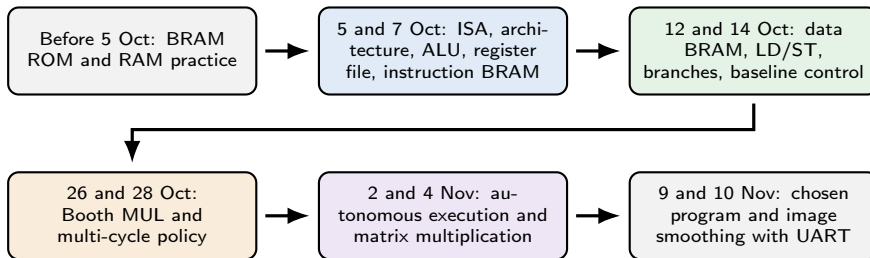
Indian Institute of Technology Kharagpur, Department of CSE

Overall Design Philosophy

- ▶ Build MiniRISC incrementally: architecture first, then datapath, memory, control, multiplier, and final programs.
- ▶ The only major new execution unit this year is the previously designed 32-bit fast Booth multiplier.
- ▶ The multiplier is not introduced in the first implementation checkpoint; it is integrated only after the baseline processor works.

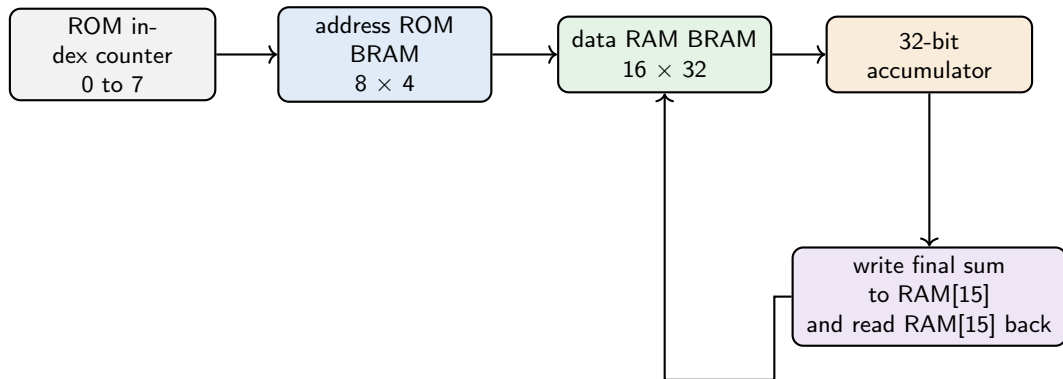


Date-wise Roadmap



- ▶ Light checkpoints are placed on the first day of a week.
- ▶ Heavier implementation demonstrations use the second day of the week.
- ▶ The final week preserves the multi-objective style: one chosen program plus mandatory convolution.

Optional Pre-sequence Practice: BRAM ROM + RAM



- ▶ Purpose: practice BRAM ROM, BRAM RAM, COE initialization and synchronous read latency.
- ▶ Expected behavioral result: accumulated value 360, stored to RAM[15] and then read back.
- ▶ This is not a processor assignment; it prepares students for instruction ROM and data RAM integration.

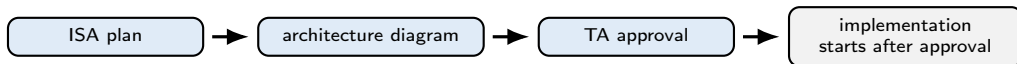
5 October: Assignment 1A - ISA Planning and Architecture

Deliverables

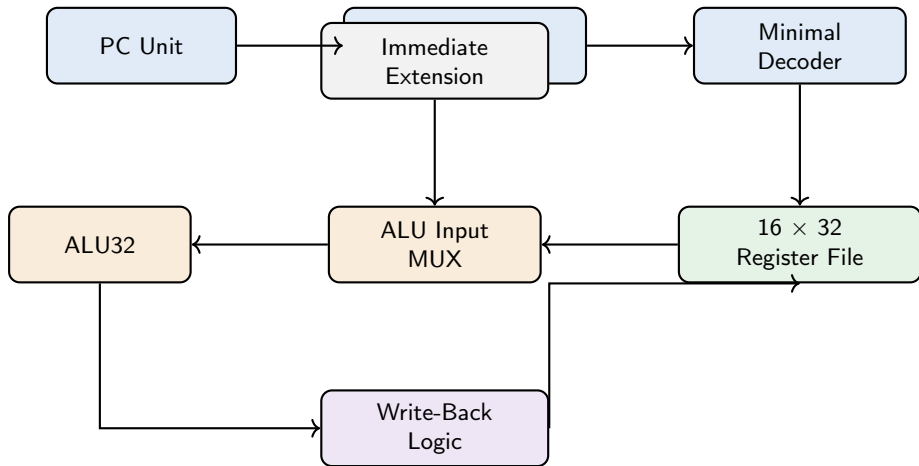
- ▶ hand-drawn processor architecture;
- ▶ clear datapath and control path separation;
- ▶ proposed instruction formats and opcode allocation;
- ▶ register, immediate, memory and branch conventions;
- ▶ future MUL or MAC instruction included in ISA planning.

Boundary

- ▶ no complete Verilog processor is required;
- ▶ multiplier appears only as a future placeholder;
- ▶ ISA freedom is allowed, but the plan must support later matrix and convolution programs.



7 October: Assignment 1B - Basic Datapath and Instruction BRAM



- ▶ Mandatory: ALU, register file, immediate extension, instruction BRAM fetch and ALU write-back.
- ▶ Instruction BRAM must be configured as single-port or dual-port ROM, as justified by the group.
- ▶ Data memory is optional here; if used, it must be BRAM RAM.

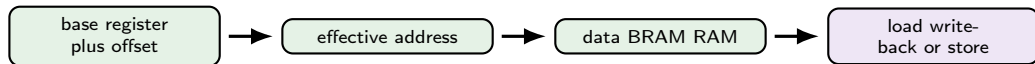
12 and 14 October: Assignment 2 - Memory and Baseline Control

12 Oct: Data BRAM + LD/ST

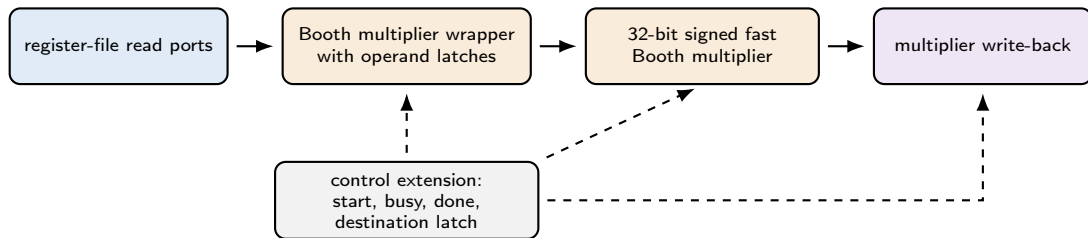
- ▶ data BRAM RAM;
- ▶ effective-address generation;
- ▶ load path from RAM to register file;
- ▶ store path from register file to RAM;
- ▶ memory write-back select.

14 Oct: Branch + baseline FSM

- ▶ branch unit;
- ▶ PC selection mux;
- ▶ NOP behavior;
- ▶ HALT behavior;
- ▶ complete baseline CPU control.



26 and 28 October: Assignment 3 - Fast Booth Multiplier Integration



- ▶ 26 Oct: integrate the multiplier wrapper and decoding for MUL/MAC.
- ▶ 28 Oct: demonstrate either hardware stall or software NOP scheduling.
- ▶ Suggested program: short dot product using repeated multiplication and accumulation.

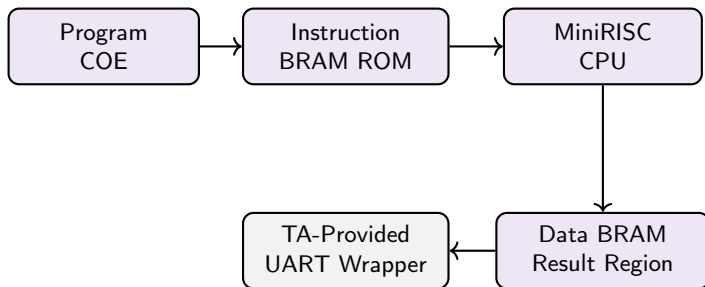
2 and 4 November: Assignment 4 - Autonomous Run and Matrix Multiplication

2 Nov: Autonomous execution

- ▶ program stored in instruction BRAM ROM;
- ▶ reset starts execution;
- ▶ PC advances automatically;
- ▶ branch instructions update PC;
- ▶ HALT freezes execution.

4 Nov: Matrix multiplication

- ▶ minimum common target: 2×2 matrix multiplication;
- ▶ use LD/ST, MUL, ADD and branches;
- ▶ store output matrix in data BRAM;
- ▶ introduce TA-provided UART wrapper.



9 and 10 November: Final Capstone

9 Nov: student-chosen program

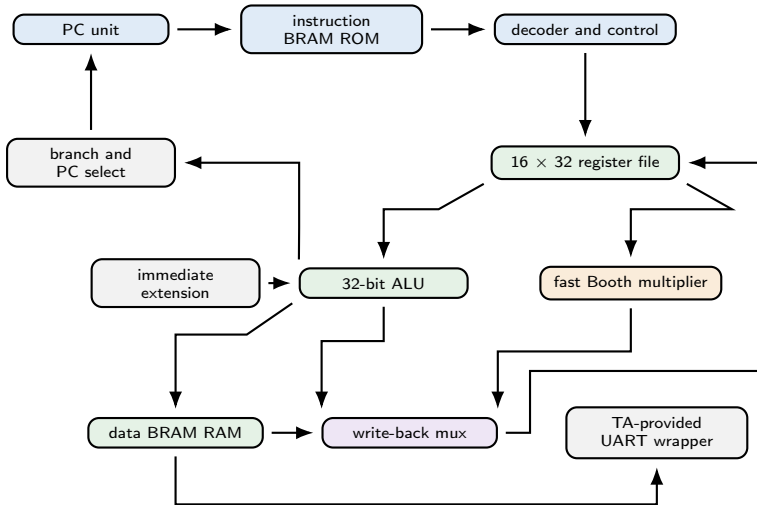
- ▶ one compact program selected by the group;
- ▶ must include loop or branch;
- ▶ must access data memory;
- ▶ must terminate using HALT.

Examples: GCD, non-restoring division, insertion sort, Hamming-weight sum or iterative factorial.

10 Nov: image-smoothing convolution

- ▶ small grayscale image preloaded in BRAM;
- ▶ fixed 3×3 kernel: $1\ 2\ 1 / 2\ 4\ 2 / 1\ 2\ 1$;
- ▶ sum of coefficients is 16, so normalization is right shift by 4;
- ▶ output pixels stored in BRAM and transmitted by UART wrapper.

Final Integrated Module View



The UART wrapper is an external verification interface used after HALT to export BRAM-resident results.

TA Checkpoints

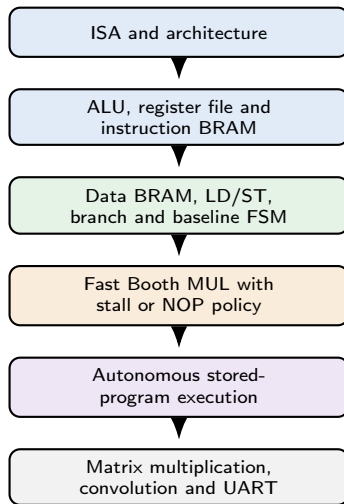
Early checkpoints

- ▶ 5 Oct: ISA and architecture coherence.
- ▶ 7 Oct: ALU, register file, instruction BRAM and write-back.
- ▶ 12 Oct: data BRAM, effective address, LD and ST.
- ▶ 14 Oct: branches, PC select, NOP/HALT and baseline FSM.

Late checkpoints

- ▶ 26 Oct: Booth multiplier reuse and start/busy/done.
- ▶ 28 Oct: stall or NOP policy with dot product.
- ▶ 2 Nov: autonomous execution and HALT.
- ▶ 4 Nov: matrix result and UART path.
- ▶ 9-10 Nov: chosen program and convolution output.

Final Dependency Chain



Each assignment should reuse validated blocks from the previous checkpoint and add only the next required processor capability.